

Policies and Permissions

Task 1

Create a custom JSON-based policy in AWS IAM that allows read-only access to all objects in an S3 bucket named 'my-music-playlists', and attach this policy to a new user called 'playlistViewer'.

Steps followed:

1. Went to IAM → Policies and clicked “Create policy”, then switched to the JSON tab to write a custom policy.
2. Wrote a policy that allows only read-related S3 actions (s3:GetObject and s3:ListBucket) so access stays strictly read-only, scoped to the ‘my-music-playlists’ bucket and its objects.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3::my-music-playlists"
    },
    {
      "Sid": "AllowReadObjects",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3::my-music-playlists/*"
    }
  ]
}
```

3. Named the policy “PlaylistViewerReadOnlyPolicy”, reviewed it, and clicked “Create policy”.
4. Went to IAM → Users → “Create user” and created a new user named “playlistViewer”.
5. On the permissions step, selected “Attach policies directly”, searched for “PlaylistViewerReadOnlyPolicy”, checked it, and completed user creation, so playlistViewer now has read-only access to the bucket.

Task 2

Write an inline policy in JSON that allows a user to start and stop EC2 instances only with the tag Environment:Dev, and attach it directly to an existing IAM user named 'devOpsIntern'. Hint: Use 'Condition' in your policy to filter by tag.

Steps followed:

1. Opened IAM → Users → devOpsIntern → “Permissions” tab → “Add permissions” → “Create inline policy”, then switched to the JSON editor.
2. Wrote a policy allowing ec2:StartInstances and ec2:StopInstances, and added a Condition block using “StringEquals” on the ec2:ResourceTag/Environment key set to “Dev”, so the actions only succeed on instances carrying that exact tag.

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```

{
  "Sid": "StartStopDevInstancesOnly",
  "Effect": "Allow",
  "Action": [
    "ec2:StartInstances",
    "ec2:StopInstances"
  ],
  "Resource": "arn:aws:ec2:*:*:instance/*",
  "Condition": {
    "StringEquals": {
      "ec2:ResourceTag/Environment": "Dev"
    }
  }
}
]
}

```

3. Named the inline policy “DevOpsInternStartStopDevOnly” and clicked “Create policy” to attach it directly to the devOpsIntern user.

4. Verified the setup by confirming devOpsIntern could start/stop an EC2 instance tagged Environment:Dev, while attempting the same action on an instance without that tag correctly failed with an “UnauthorizedOperation” error.

Task 3

Create a managed policy in AWS IAM that grants permission to list and view DynamoDB tables, then attach this policy to a group called 'appReviewers' which could represent users reviewing app data like Zomato order history.

Steps followed:

1. Went to IAM → Policies → “Create policy” → JSON tab, and wrote a policy allowing dynamodb:ListTables, dynamodb:DescribeTable, and dynamodb:GetItem/Query so reviewers can list and view table contents.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ViewDynamoDBTables",
      "Effect": "Allow",
      "Action": [
        "dynamodb:ListTables",
        "dynamodb:DescribeTable",
        "dynamodb:GetItem",
        "dynamodb:Query"
      ],
      "Resource": "*"
    }
  ]
}

```

2. Named the policy “AppReviewersDynamoDBViewOnly”, reviewed it, and clicked “Create policy” so it is saved as a reusable managed policy.

3. Went to IAM → User groups → “Create group” and named the group “appReviewers”.

4. On the “Attach permissions policies” step, searched for and selected “AppReviewersDynamoDBViewOnly”, then clicked “Create group”, so any user placed in this group (for example, a team member reviewing Zomato-style order history data) automatically gets read-level access to DynamoDB tables.

Task 4

Given the following JSON policy, identify and fix the error that would prevent it from granting access as intended. Hint: Check if the policy allows listing the bucket as well as getting objects.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::user-uploads/*"
    }
  ]
}
```

Error identified:

The policy only grants s3:GetObject on the objects inside the bucket (“arn:aws:s3:::user-uploads/*”), but it never grants s3:ListBucket on the bucket itself (“arn:aws:s3:::user-uploads”). Without s3:ListBucket, the user cannot browse or list the contents of the bucket in the console or via “aws s3 ls”, even though they can fetch an object if they already know its exact key. This is a common mistake, since ListBucket is a bucket-level action and needs the bucket ARN (no trailing /*), while GetObject is an object-level action and needs the object ARN (with /*) — the two cannot share one Resource line.

Corrected policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowListBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::user-uploads"
    },
    {
      "Sid": "AllowGetObject",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::user-uploads/*"
    }
  ]
}
```

Task 5

Use ChatGPT to generate a JSON IAM policy that allows users to only update their own profile information in a DynamoDB table named 'UserProfiles', then review and test the policy in the AWS Policy Simulator. Hint: Ask ChatGPT for a policy with a condition that enforces users can update only their own data.

Steps followed:

1. Asked ChatGPT to generate an AWS IAM JSON policy that allows a user to update only their own item in a DynamoDB table named 'UserProfiles', where each item's partition key matches the logged-in IAM username.
2. The AI tool returned a policy using dynamodb:UpdateItem with a Condition block based on the "dynamodb:LeadingKeys" condition key, tied to the "\${aws:username}" policy variable, so the condition is only satisfied when the partition key of the item being updated equals the requester's own username.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowUpdateOwnProfileOnly",
      "Effect": "Allow",
      "Action": "dynamodb:UpdateItem",
      "Resource": "arn:aws:dynamodb:*:*:table/UserProfiles",
      "Condition": {
        "ForAllValues:StringEquals": {
          "dynamodb:LeadingKeys": [
            "${aws:username}"
          ]
        }
      }
    }
  ]
}
```

3. Reviewed the generated policy for correct JSON syntax and confirmed that the partition key of the 'UserProfiles' table is the username, which is required for the LeadingKeys condition to work correctly.
4. Opened the AWS IAM Policy Simulator, selected the user this policy was attached to, chose the dynamodb:UpdateItem action on the UserProfiles table, and tested it once with the resource matching the user's own key (simulation returned "allowed") and once with a different user's key (simulation returned "implicitly denied"), confirming the condition works as intended.